

Library Books API Assignment

Goal

Build a simple REST API using **Node.js**, **Express**, and **MongoDB** to manage books in a library.

Instructions

Step 1: Project Setup

1. Create a new project folder.
2. Initialize a Node project using `npm init -y`.
3. Install dependencies:
 - a. Express=> `npm install express`
 - b. Mongoose=> `npm install mongoose`
 - c. Nodemon =>`npm install nodemon`
 - d. Dotenv => `npm install dotenv`
4. Update package.json so the server runs with nodemon using the command:

npm run dev or npm start

5. Add the module's type at package.json

type : "module"

6. Ensure **server.js** is the main file of the project.

Step 2: MongoDB Setup at server.js and .env

1. Set up a MongoDB URI at .env
2. Add a database endpoint called **libraryDB** at the end of the connection string at the .env file

e.g

[MONGO_URI=mongodb+srv://studentadmin:StrongPass123@cluster0.hvjtkfs.mongodb.net/LibraryDB](#)

3. Create a Book schema with these fields:
 - a. title (required, string)
 - b. author (required, string)
 - c. yearPublished (number)
 - d. genre (string)
 - e. available (boolean, default: true)

Step 3 : Server Setup (server.js)

1. Import Express and Mongoose.
2. Connect to **libraryDB**.
3. Add middleware to parse JSON requests.
4. Import book routes from a routes folder.
5. Attach all routes to the base path **/books**.
6. Start the server on a port (e.g., 3000, 2000, etc.).

Step 4: Routing Structure

1. Create a routes folder.
2. Inside it, create a books.js route file.
3. Implement the following API endpoints:

Required Routes

- a. **GET /books**
Returns a list of all books.

- b. **GET /books/:id**
Returns a single book by ID.
- c. **POST /books**
Creates a new book record.
- d. **PUT /books/:id**
Updates the details of a selected book.
- e. **DELETE /books/:id**
Deletes a selected book from the database.

Step 5: Testing with Insomnia

Test ALL endpoints in **Insomnia**:

1. Create a new POST request to /books
 - a. Send JSON data for a new book.
2. Create a GET request to /books
 - a. Confirm that the book created appears.
3. Create a GET request to /books/:id
 - a. Confirm that a single book can be fetched.
4. Create a PUT request to /books/:id
 - a. Update fields such as available or genre.
5. Create a DELETE request to /books/:id
 - a. Confirm that the record is removed.